What happens when an LLM generates a response?

A Large Language Model, or LLM, generates answers based on patterns it learned during training. During training, the model reads a very large amount of text and learns how words, phrases, concepts, and ideas are related.

At a very high level, when a user asks a question, the LLM does not search the internet or open documents by default. Instead, it uses:

the knowledge stored in its trained parameters,
the current prompt,
and the conversation context

to predict the most likely next words and form a response.

Simple way to think about it

An LLM is like a very knowledgeable person who has read a huge library in the past, but at the moment of answering, that person is usually speaking from memory, not by checking a live source.

Example

If you ask:

“What is Java?”

The LLM can answer based on what it learned during training:

- Java is a programming language
- It is object-oriented
- It is widely used in backend systems, enterprise applications, Android development, etc.

This answer may be correct because this is common knowledge that the model has likely seen many times during training.

Important points

LLMs are very good at:

- language generation,
- summarization,
- explanation,
- paraphrasing,
- question answering based on prior knowledge.

But they are not automatically connected to:

- your company documents,
- private PDFs,
- internal databases,
- latest real-time updates,
- or live external systems.

Limitations of LLM knowledge

Although LLMs are powerful, their knowledge has important limitations.

Static knowledge

The model is trained on data from the past. Its internal knowledge is not updated every second.

So if something changes after training:

- a company policy,
- stock price,
- product catalog,
- medical guideline,
- government rule,
- or a new internal document,

the LLM may not know it.

Example

Suppose a company changed its leave policy last week from 15 days to 20 days.
A normal LLM may still answer with the old information if it only relies on pretrained memory.

Lack of domain-specific context

A general-purpose LLM may know public information well, but it does not automatically know your private or business-specific knowledge.

Examples:

- your organization's HR handbook,
- internal engineering wiki,
- customer support manuals,
- project architecture documents,
- legal agreements,
- medical records,
- product requirement documents.

Example

If a student asks:

“What are the grading rules in our institute?”

A general LLM cannot answer correctly unless those rules are provided in the prompt or retrieved from the institute’s documents.

No built-in access to external or real-time data

By default, an LLM does not fetch fresh information from a database, document store, or search engine unless a system is specifically built to do that.

This means it may fail on questions like:

- “What is the current refund policy?”
- “What is the latest version of our API?”
- “Which document mentions Project Falcon?”
- “What was today’s ticket count?”
- “What is the updated onboarding process?”

Answers may sound confident even when wrong
This is one of the biggest challenges. Because the model is designed to generate fluent text, it can produce an answer that:

- sounds natural,
- sounds confident,
- sounds complete,

but it is still wrong.

That leads us to **`hallucination`**.

What is hallucination in LLMs?

Hallucination means the model generates information that is:

- incorrect,
- fabricated,
- unsupported,
- or misleading.

The model may invent:

- facts,
- references,
- policy details,
- citations,
- names,
- numbers,
- or explanations.

Why does hallucination happen?

Usually because:

- the model does not actually know the answer,
- the prompt does not provide enough information,
- the model tries to fill the gaps using learned language patterns.

Example 1

User asks:

“What does our company’s 2026 reimbursement policy say about certification courses?”

If the LLM has not seen that document, it may still generate something like:

“Employees are eligible for up to ₹50,000 per year for certification reimbursements.”

This may sound professional, but it could be completely made up.

Example 2

A student uploads a PDF and asks:

“What does page 12 conclude?”

If the system does not actually retrieve or read page 12, the LLM may still guess and produce a false summary.

Key idea

Hallucination is dangerous because the answer often looks correct, even when it is not.

Why do we need grounded AI systems?

For many real-world applications, “sounding good” is not enough. We need systems that are accurate, reliable, and based on real sources.

This is where grounding becomes important.

Grounded AI means:

The model’s response should be based on external, verifiable information, not only on internal memory.

A grounded AI system should ideally answer using:

- retrieved documents,
- database records,
- company policies,
- product manuals,
- support articles,
- medical references,
- legal texts,
- or any trusted knowledge source.

Real-life analogy
Imagine two employees answering a question:

- Employee A answers from memory only.
- Employee B first checks the official document, then answers.

Which answer is more trustworthy?
Obviously, Employee B.

That is the difference between:

- an LLM-only approach
- and a grounded system like RAG

What is Retrieval-Augmented Generation (RAG)?

RAG stands for Retrieval-Augmented Generation.

It is an approach where the system:

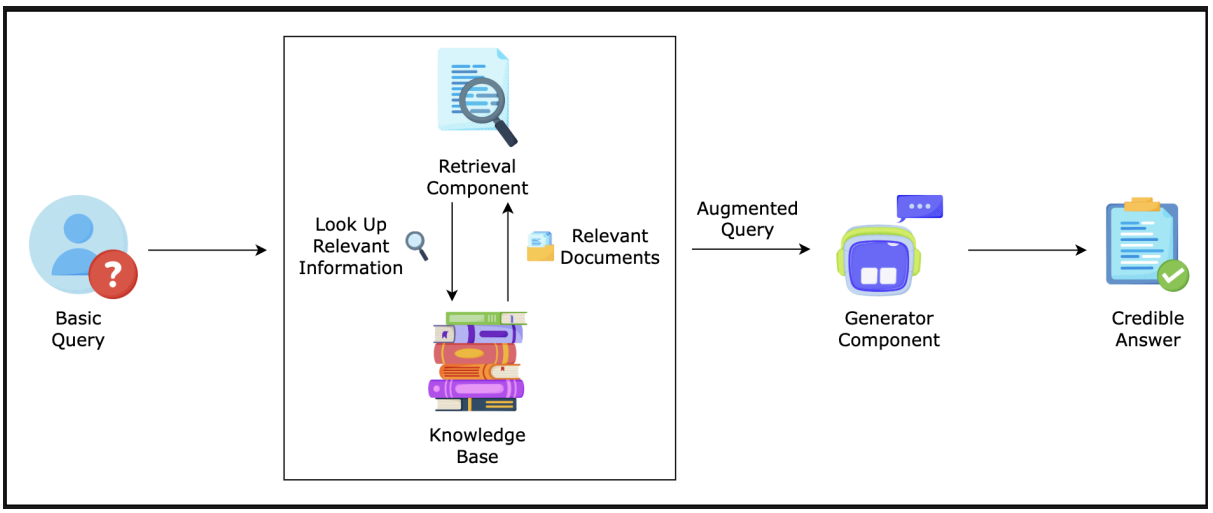
- takes the user's query,
- retrieves relevant information from external knowledge sources,
- provides that information to the LLM as context,
- and then the LLM generates the answer using that retrieved context.

Simple definition

RAG is a method that combines:

retrieval of relevant information with generation of natural language responses.

So instead of answering only from memory, the model answers using retrieved evidence.

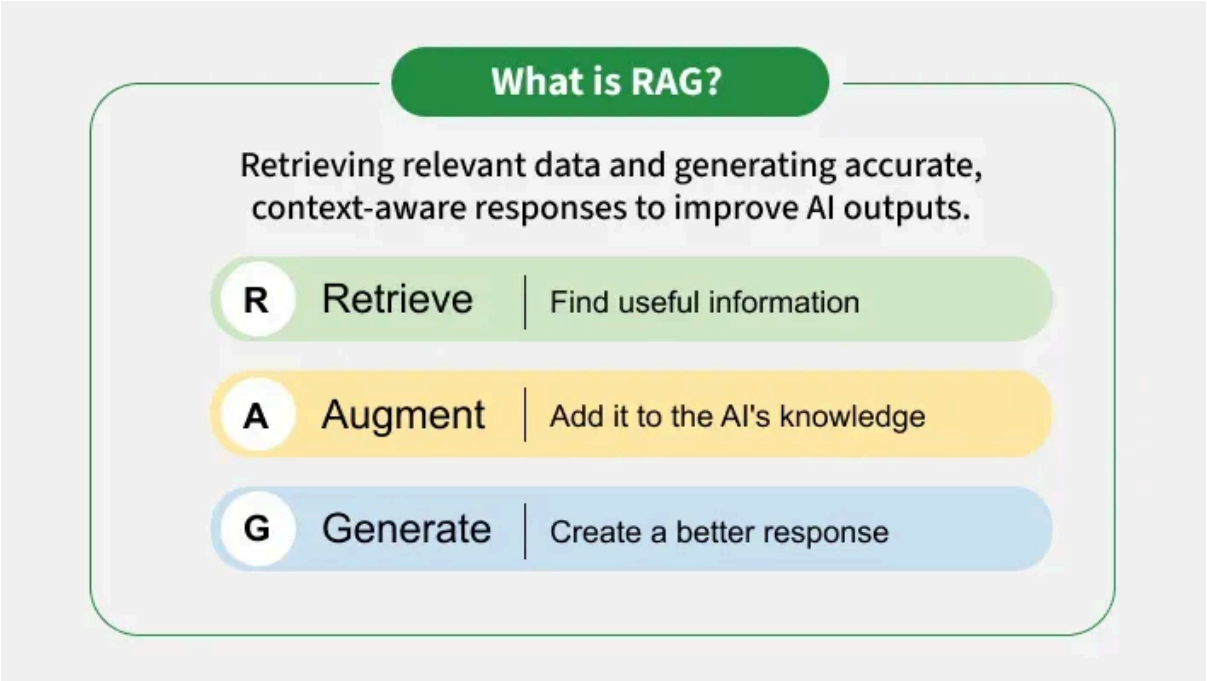


Why is RAG important?

RAG helps solve the main limitations of plain LLMs.

It improves:

- accuracy,
- relevance,
- trustworthiness,
- domain awareness,
- and explainability.



Without RAG

The model answers from general pretrained knowledge.

With RAG

The model answers using specific retrieved documents or knowledge.

Example

User asks:

“What is the leave encashment policy in our company?”

LLM-only response

The model might give a generic HR-style answer.

RAG-based response

The system first retrieves the HR policy document, finds the section on leave encashment, and then generates an answer based on that exact content.

That makes the answer much more useful and reliable.

What does “grounding” mean in RAG?

Grounding means anchoring the model’s answer in actual retrieved information.

In RAG, the LLM is not expected to “invent” the answer. Instead, it is expected to use the retrieved content as support.

Example: Suppose a retrieved document says:

“Employees may carry forward up to 10 unused leave days to the next calendar year.”

A grounded response would be:

“According to the policy document, employees can carry forward up to 10 unused leave days.”

This is far better than a generic guess.

Why does grounding matter ?

Grounding:

- reduces hallucination,
- improves factual correctness,
- makes responses more domain-specific,
- makes it easier to trace the source of the answer.

Grounding means the model is answering from evidence, not from imagination.

High-level flow of RAG

The basic RAG flow is:

Query → Retrieve relevant information → Provide context to LLM → Generate response

Let us understand each step clearly.

Step 1: User asks a query
This is the input question from the user.

Example:

“What is the refund rule for damaged products?”

Step 2: System retrieves relevant information

The system searches its knowledge base to find the most relevant documents, chunks, or passages.

The knowledge source may be:

- PDFs
- company documents
- FAQs
- database records
- wiki pages
- product manuals
- knowledge base articles

Example:
The retrieval system finds a policy chunk that says:

“Damaged products reported within 48 hours of delivery are eligible for replacement or refund after verification.”

Step 3: Retrieved content is given to the LLM as context

The model now receives:

- the user’s question
- and the retrieved text

So the LLM is not answering blindly. It now has supporting material.

Step 4: LLM generates the final answer

The LLM uses the retrieved content to produce a readable, natural-language response.

Example:

“According to the policy, damaged products reported within 48 hours of delivery may qualify for a replacement or refund after verification.”

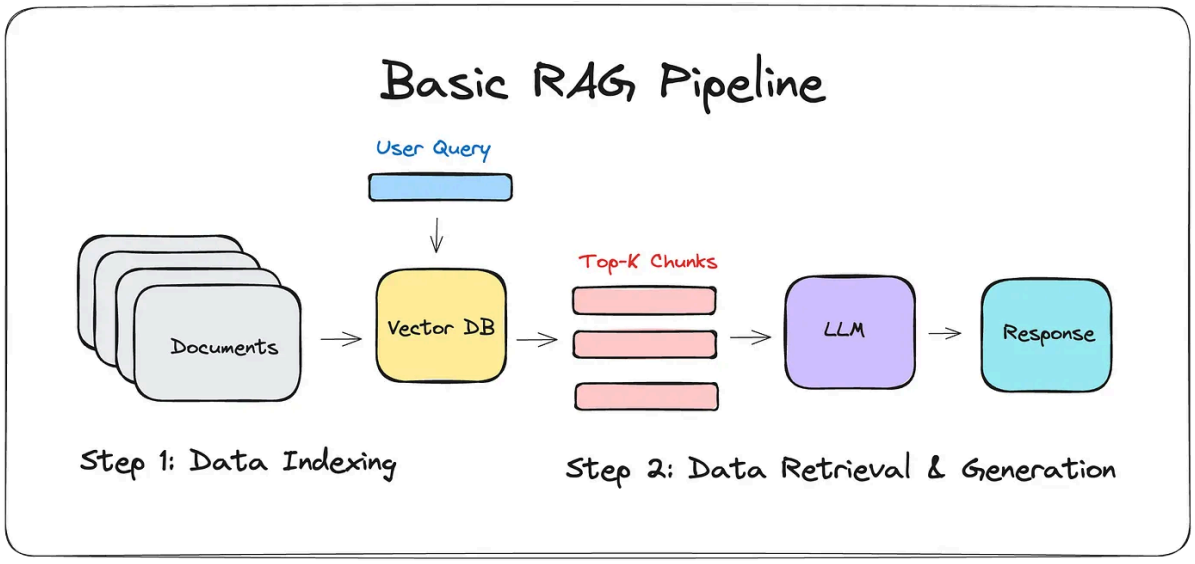
Flow Diagram

```
User Query
   ↓
Retrieve relevant documents/chunks
   ↓
```

```
Pass retrieved context to LLM
   ↓
LLM generates grounded answer
```

LLM-only vs RAG-based responses

Aspect	LLM-only	RAG-based
Knowledge source	Pretrained memory	Pretrained memory + retrieved external data
Access to latest info	Usually no	Yes, if knowledge base is updated
Domain-specific accuracy	Limited	Much better
Hallucination risk	Higher	Lower
Traceability	Weak	Stronger
Usefulness for enterprise data	Poor to moderate	High



Example comparison
Query: “What is the latest onboarding process for new employees in our company?”

LLM-only answer

The model may give a generic onboarding process:

- account creation,
- document verification,

- team introduction,
- training sessions.

This may sound fine, but it may not match the company's actual process.

RAG-based answer

The system retrieves the latest HR onboarding guide and then answers:

- pre-joining document submission,
- laptop collection process,
- internal tool setup,
- manager introduction meeting,
- mandatory security training,
- payroll verification steps.

This answer is specific and grounded.

How does RAG connect to vector search systems?

This is a key concept because most modern RAG systems use vector search for retrieval.

What is the role of retrieval in RAG?

The retrieval part must find the most relevant pieces of information for the user's query.

Traditional keyword search can help in some cases, but it has limitations:

- it may miss meaning,
- it may fail when the words are different but the intent is similar,
- it may not work well for semantic similarity.

That is why vector search is commonly used.

What is vector search in simple words?

Vector search converts text into numerical representations called embeddings.

These embeddings capture semantic meaning.

Then the system compares:

the embedding of the user query with the embeddings of stored document chunks

and finds the most semantically similar ones.

Example

Suppose a document says:

“Employees can work remotely for up to two days each week.”

User asks:

“What is our work from home policy?”

Even if the exact words “work from home policy” are not written in the document, vector search can still retrieve the correct chunk because the meanings are similar.

Why is vector search useful for RAG ?

Vector search helps RAG retrieve context that is:

- semantically relevant,
- not just keyword matched,
- useful even when wording differs.

This improves the quality of the final response.

Simple pipeline with vector search:

```
Documents
   ↓
Convert into chunks
   ↓
Create embeddings
   ↓
Store in vector database
   ↓
User query → create query embedding
   ↓
Retrieve similar chunks
   ↓
Pass chunks to LLM
   ↓
Generate answer
```

Real-world use cases of RAG

RAG is very valuable when answers need to come from specific knowledge sources.

Enterprise knowledge assistants

Companies often have thousands of internal documents:

- HR policies
- engineering documentation
- architecture notes
- onboarding guides
- compliance documents
- internal FAQs

Employees waste time searching manually.

A RAG-based assistant can answer questions like:

- “What is the reimbursement policy?”
- “How do I raise a production access request?”
- “Where is the deployment checklist documented?”
- “What is the leave carry-forward rule?”

Why is RAG useful here ?

Because the answers must come from internal documents, not general internet knowledge.

Document-based question answering

Users often need answers from:

- PDFs
- contracts
- research papers
- invoices
- policy documents
- manuals
- legal documents

Instead of reading the entire document, users can ask:

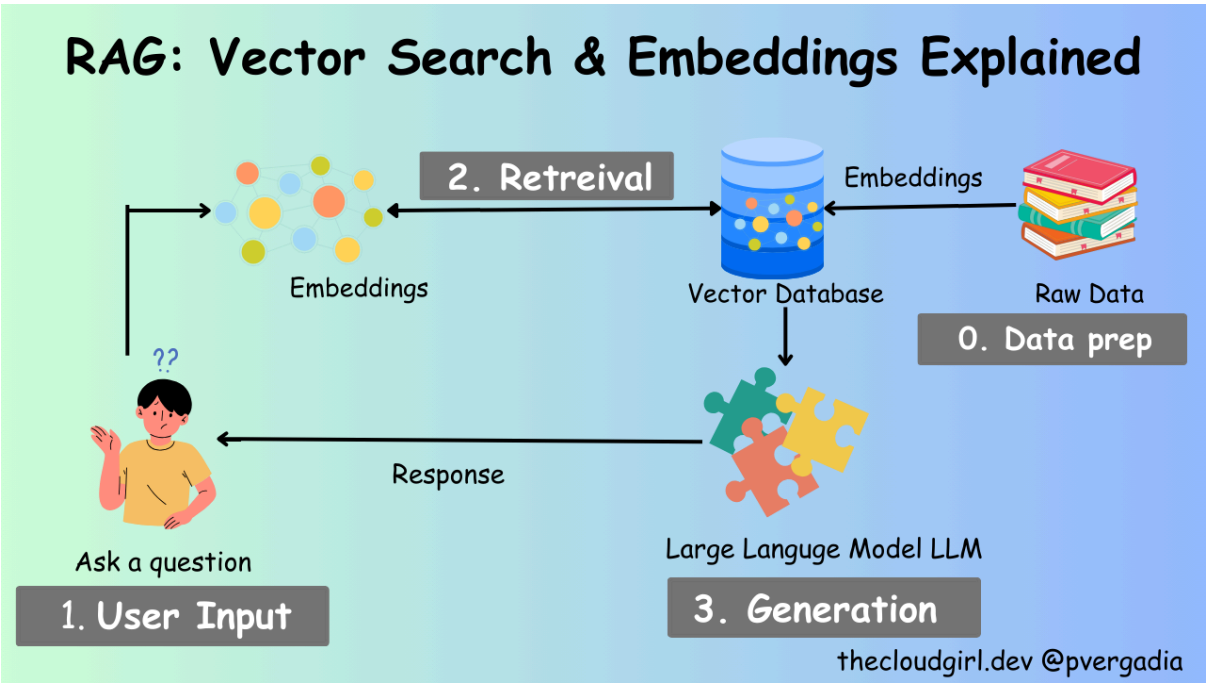
- “What is the termination clause?”
- “What is the refund condition?”
- “What does this report conclude?”
- “What are the warranty terms?”

The RAG system retrieves the relevant section and answers from it.

Example: A student uploads lecture notes and asks:

“What are the three main benefits of indexing mentioned in the notes?”

The system retrieves the relevant part of the notes and answers directly.



RAG in agentic systems

Now let us connect RAG with AI agents.

What is an agent in simple terms?

An AI agent is a system that can:

- understand a goal,
- reason about what to do,
- use tools,
- access information,
- and take action.

In many cases, an agent should not rely only on memory. It needs access to current knowledge.

That is where retrieval becomes important.

Why do agents need retrieval ?

Suppose an agent is helping with:

- customer support,
- internal operations,
- document analysis,
- software troubleshooting,

- travel planning,
- policy lookup.

The agent needs to make decisions based on actual information, not just guesswork.

Example: A support agent gets this request:

“Can a customer return a product after 15 days if the product is defective?”

If the agent retrieves the latest return policy first, then its reasoning and final action become more reliable.

RAG improves agent decision-making

When an agent retrieves relevant knowledge before responding or acting, it can:

- make better decisions,
- reduce mistakes,
- justify its responses,
- adapt to domain-specific rules,
- and work with updated information.

Agentic Flow:

```
User goal
   ↓
Agent decides it needs information
   ↓
Agent retrieves relevant knowledge
   ↓
Agent reasons over retrieved context
   ↓
Agent responds or takes action
```

Example

An internal IT helpdesk agent receives:

“How do I request VPN access?”

The agent first retrieves the latest IT access procedure, then gives the correct steps.

Without retrieval, it may provide outdated or incorrect instructions.

Why is retrieval quality so important ?

A RAG system is only as good as its retrieval.

This is a very important teaching point.

Even if the LLM is strong, poor retrieval leads to poor responses.

Why?

Because the final answer depends on the retrieved context.

If retrieval returns:

- irrelevant chunks,
- incomplete chunks,
- outdated chunks,
- or no useful chunks,

then the LLM may:

- answer incorrectly,
- answer incompletely,
- hallucinate,
- or produce vague output.

```
Better retrieval → better context → better response quality
```

User asks:

“What is the approval process for travel reimbursement?”

Good retrieval

The system finds the latest travel policy and reimbursement workflow.
Result: accurate answer.

Poor retrieval

The system retrieves an unrelated finance document or an old outdated policy.
Result: confusing or incorrect answer.

What affects retrieval quality?

At a high level, retrieval quality depends on:

- how documents are chunked,

- how embeddings are created,
- how search is performed,
- how relevant chunks are ranked,
- whether metadata is used,
- whether the knowledge base is updated.

You do not need deep technical details in an introduction class, but students should understand that retrieval quality directly impacts answer quality.
Let us take one complete example.

Scenario

A company has an HR policy document.

The user asks:

“How many casual leaves can employees take in a year?”

Without RAG

The LLM may respond:

“Most companies allow 10 to 12 casual leaves per year.”

This is a generic guess. It may be wrong for that company.

With RAG

Step 1: Query

The user asks the question.

Step 2: Retrieval

The system searches the HR policy and retrieves this chunk:

“Full-time employees are entitled to 8 casual leave days per calendar year.”

Step 3: Context to LLM

This retrieved chunk is passed to the model.

Step 4: Generation

The LLM responds:

“According to the HR policy, full-time employees are entitled to 8 casual leave days per calendar year.”

This answer is:

- specific,
- grounded,
- and reliable.